Especificação Técnica e Governança: TaskAnalyzer

Paradigma: Spec-Driven Development (SDD) & AI Harness Governança
Linguagem: Python 3.11+

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação

- Lucas Martins de Andrade
- Ciência da Computação
- Asa Norte / Noturno
- lucas.ma@sempreceub.com

1.2 Propósito do Módulo TaskAnalyzer

O **TaskAnalyzer** é um módulo avançado de análise de produtividade e métricas de desempenho de tarefas no ciclo de desenvolvimento de software. Atuando como o motor de inteligência analítica do sistema, o módulo processa históricos e coleções de tarefas para gerar indicadores consolidados e acionáveis sobre:

- **Tempo médio de conclusão:** Duração média em dias das tarefas efetivamente finalizadas.
- **Taxa de atraso:** Percentual de tarefas que excederam o prazo estipulado (seja em entregas concluídas fora do prazo ou em tarefas em aberto cujo prazo já expirou em relação à data de referência).
- **Indicadores por prioridade:** Segmentação detalhada dos indicadores por categorias de prioridade (ALTA, MÉDIA, BAIXA).

1.3 Contrato Executável de Interface

No paradigma **Spec-Driven Development (SDD)**, o contrato técnico é a fonte única da verdade (*Single Source of Truth*) e define rigorosamente a estrutura dos dados de entrada, os dados de saída e as assinaturas de funções executáveis.

Modelos de Dados (Dataclasses e Enums)

```
from dataclasses import dataclass
from datetime import datetime
from enum import Enum
from typing import List, Dict, Optional
```

```
class PrioridadeEnum(str, Enum):
    ALTA = "ALTA"
    MEDIA = "MEDIA"
    BAIXA = "BAIXA"

@dataclass(frozen=True)
class Task:
    """Representa uma tarefa individual no sistema."""
    id: str
    titulo: str
    data_criacao: datetime
    prazo_estimado: datetime
    prioridade: PrioridadeEnum
    data_conclusao: Optional[datetime] = None

@dataclass(frozen=True)
class MetricByPriority:
    """Métricas consolidadas para um nível de prioridade específico."""
    total_tarefas: int
    concluidas: int
    atrasadas: int
    tempo_medio_conclusao_dias: float

@dataclass(frozen=True)
class MetricsResult:
    """Resultado consolidado da análise de métricas de produtividade."""
    total_tarefas: int
    total_concluidas: int
    taxa_atraso_percentual: float
    tempo_medio_conclusao_dias: float
    metricas_por_prioridade: Dict[PrioridadeEnum, MetricByPriority]
```

Assinatura do Módulo Principal

```
def analyze_tasks(tasks: List[Task], data_referencia: Optional[datetime] = None) -> MetricsResult:
    """
    Processa a lista de tarefas e retorna os indicadores consolidados de produtividade.

    Args:
        tasks: Lista de objetos Task a serem analisados.
        data_referencia: Ponto no tempo utilizado como referência para checar atrasos
                         em tarefas em aberto. Se None, utiliza datetime.now().

    Returns:
```

```
        MetricsResult contendo as métricas consolidadas gerais e por prioridade.

    Raises:
        EmptyTaskListError: Se a lista de tarefas fornecida estiver vazia.
        InvalidTaskDateError: Se a data de conclusão for anterior à data de criação.
    """
    pass
```

Especificação Técnica dos Atributos e Tipos

Estrutura / Campo	Tipo Python	Obrigatório?	Descrição / Regra de Negócio
Task.id	str	Sim	Identificador único da tarefa.
Task.data_criacao	datetime	Sim	Data/hora de registro da tarefa.
Task.prazo_estimado	datetime	Sim	Data/hora limite estipulada para entrega.
Task.data_conclusao	Optional[datetime]	Não (None se aberta)	Data/hora de encerramento efetivo. Deve ser >= data_criacao.
MetricsResult.taxa_atraso_percentual	float	Sim	Valor percentual entre 0.0 e 100.0.
MetricsResult.tempo_medio_conclusao_dias	float	Sim	Média em dias das tarefas concluídas. Retorna 0.0 se nenhuma concluída.

2. Especificação de Cenários de Aceite e Test Harness

2.1 Cenários de Teste (BDD: Dado-Quando-Então)

Os cenários de aceite são definidos de forma declarativa e servem como a base inviolável do Test Harness.

Cenário 1 (Caminho Feliz / Sucesso): Processamento com Métricas Válidas

- **Dado** uma lista contendo 3 tarefas com os seguintes dados:
    - *Tarefa 1:* Prioridade ALTA, criada há 5 dias, prazo em 3 dias, concluída em 2 dias (Sem atraso).
    - *Tarefa 2:* Prioridade ALTA, criada há 10 dias, prazo há 2 dias, concluída há 1 dia (Atrasada em 1 dia).
    - *Tarefa 3:* Prioridade MÉDIA, criada há 4 dias, prazo há 1 dia, não concluída (Atraso ativo).
- **Quando** o analisador analyze_tasks for executado fornecendo a lista acima,
- **Então** deve retornar um objeto MetricsResult onde:
    - total_tarefas é igual a 3.
    - total_concluidas é igual a 2.
    - tempo_medio_conclusao_dias é igual a 1.5 dias (média de 2 dias e 1 dia).
    - taxa_atraso_percentual é igual a 66.67% (2 tarefas atrasadas em 3 no total).
    - metricas_por_prioridade[PrioridadeEnum.ALTA].atrasadas é igual a 1.
    - metricas_por_prioridade[PrioridadeEnum.MEDIA].atrasadas é igual a 1.

Cenário 2 (Tratamento de Exceção): Inconsistência Temporal nas Datas

- **Dado** uma lista de tarefas contendo uma tarefa onde a data_conclusao é 2026-01-01 e a data_criacao é 2026-01-05,
- **Quando** a função analyze_tasks for chamada com este parâmetro,
- **Então** o sistema deve interromper a execução e disparar a exceção customizada InvalidTaskDateError com mensagem clara.

Cenário 3 (Tratamento de Exceção): Entrada de Lista Vazia

- **Dado** uma lista de tarefas vazia ([]),
- **Quando** a função analyze_tasks for executada,
- **Então** o sistema deve disparar a exceção customizada EmptyTaskListError.

2.2 Integração do Test Harness com pytest

No modelo de desenvolvimento assistido por IA, o desenvolvedor não valida o código manualmente. A suíte de testes automatizados atua como o homologador automático do código gerado pela IA.

1. **Tradução BDD para pytest:** Cada cenário especificado no documento vira uma função de teste automatizado no arquivo tests/test_harness.py.
2. **Ciclo de Feedback Automatizado:** O Agente de IA recebe os testes e executa pytest tests/test_harness.py para verificar a conformidade de sua implementação.
3. **Aprovação Rígida:** O código gerado pela IA só é considerado pronto quando 100% dos cenários do harness passarem sem falhas e sem violações de cobertura.

3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES.md)

As regras de contexto formam a camada de governança (*AI Harness*) que impede o desalinhamento arquitetural e a proliferação de alucinações de código.

3.1 Diretrizes Arquiteturais Obrigatórias	3.2 Proibições Explícitas (Guardrails)
• **Versão Mínima:** Python 3.11+. • **Tipagem Estrita:** Type hints obrigatórios em 100% das funções e assinaturas. • **Código Limpo:** Adesão rigorosa ao PEP 8 e princípio SRP (Single Responsibility Principle). • **Documentação Formal:** Google Style Docstrings em todas as funções públicas e classes. • **Exceções Customizadas:** Mensagens explícitas com exceções dedicadas. • **Logs Estruturados:** Uso do módulo nativo logging para rastreabilidade de execução.	• **Sem Bibliotecas Externas Não Autorizadas:** Proibido importar bibliotecas de terceiros (como Pandas, NumPy) sem aprovação. • **Imutabilidade dos Testes:** Proibido alterar, remover ou simplificar cenários do test_harness.py. • **Sem Persistência de Dados:** Proibido criar arquivos, banco de dados local ou comunicação de rede. • **Imutabilidade de Contrato:** Proibido alterar nomes, parâmetros ou tipos das funções estipuladas no contrato. • **Sem Código Duplicado:** Reutilização e código modular obrigatórios.

4. Arquitetura do Repositório e Estrutura de Arquivos

A organização do repositório no Git/GitHub segue o padrão modular estabelecido para o ecossistema SDD:

```
📁 root / (repositório-principal)
│
├── 📁 .context /                        (Diretório de Governança e Regras)
│   ├── 📄 CONTEXT_RULES.md     Diretrizes e limites de engenharia p/ IA)
│   └── 📁 contracts /            (Contratos de negócio e especificações)
│
├── 📁 src /                                  (Diretório de Código-Fonte)
│   ├── 📁 modules /             (Módulos de domínio e regras de negócio)
│   └── 📁 shared /             (Utilitários, erros e componentes comuns)
│
├── 📁 tests /                            (Diretório de Testes e Validação)
│   ├── 📁 unit / & integration / (Suíte completa de testes de regressão)
│   └── 📁 edge_cases /           (Casos limite para validação manual/IA)
│
└── 📁 .github /                         (Diretório de CI/CD e Repositório)
    ├── 📁 workflows /          (Pipelines automatizadas de CI e Linters)
    └── 📄 PULL_REQUEST_TEMPLATE.md         (Checklist de revisão humana)
```